

Database Schema and Idempotency Design

<https://gemini.google.com/app/5712936e69ab0063>

User prompt: Starter schema: locals: id, name members: id, local_id, full_name, email, classification, status announcements: id, local_id, title, body, needs_ack, sent_at, created_at I need to add a per recipient delivery record, sent, read, acknowledged, and separately an optional RSVP, plus a way to tell a leadership account from a member account for authorization. Here is the call I am making, tell me if it is wrong. Model RSVP as three states: coming, cant, no_response as the default. Not free text and not a boolean, because leadership's real need, knowing who is coming so they are not standing in an empty room, is a headcount, and three fixed states make that a single fast query instead of parsing text. Role: a role field on members, leadership or member, instead of a separate leadership table, because a leader is still tied to one local and does not need different core fields. Tell me if either call is wrong, then give me the full schema including the recipient table." Prompt 6, idempotency first, not added later: "There are 22,400 recipients per send, and a hard requirement that a retried request, or a process that crashes and restarts mid send, must never deliver twice to the same member. I want that guarantee to live in a database constraint, not in application code that checks for an existing row before inserting one, because that kind of check can race under concurrent retries. Plan: a unique constraint on announcement_id and member_id in the recipient table, and fan out done with INSERT ... ON CONFLICT DO NOTHING, so a retried batch simply does nothing for rows that already exist, no matter which process or server handles it. Confirm this actually holds under retries and crash restarts, or tell me where it breaks

Response: Evaluation of Proposed Calls & Schema Design

1. RSVP & Role Design Evaluation

- **RSVP States (coming, cant, no_response): Correct.** Storing RSVP as an explicit enumerated or constrained state (coming, cant, no_response) instead of free-text or a boolean is the correct design. A boolean cannot distinguish between an explicit decline ("cant") and someone who has not seen or responded to the message ("no_response"). Using fixed enum values enables fast, indexable SQL aggregation (COUNT ... GROUP BY rsvp) for real-time headcount queries.
- **Role Field on members: Correct.** Since a leader is bound to a single local and shares core attributes (e.g., contact info, classification) with regular members, adding a role field (or is_leadership boolean/enum) directly on members avoids unnecessary JOIN overhead and table fragmentation.

Evaluation of Idempotency & Concurrency Plan (Prompt 6)

Where the Assumption Holds

Under normal execution and retries (e.g., standard API retries or network timeout re-transmissions), a UNIQUE(announcement_id, member_id) constraint paired with INSERT ... ON CONFLICT DO NOTHING guarantees that no duplicate recipient rows will be inserted.

Where It Breaks (Edge Cases & Failure Modes)

1. Partial Success During Crashes (Incomplete Fan-out):

- If a background worker processing a batch of 22,400 recipients crashes mid-way (e.g., after inserting 10,000 rows), a naive retry of the entire process using INSERT ... ON CONFLICT DO NOTHING will safely skip the first 10,000 rows and insert the remaining 12,400.
- **The Break Point:** While the *database state* remains idempotent, the **external side-effect (push notification / SMS dispatch)** is not guaranteed to be idempotent unless execution is explicitly staged. If notification dispatch occurs in the same loop as row insertion without atomic status

tracking, retried batches could attempt to re-send actual messages or skip members who failed to receive them.

2. Delivery Status vs. Database Row Creation:

- If the worker inserts all 22,400 recipient rows in a single batch before sending notifications, and then crashes mid-send, a retry will see that all 22,400 rows exist in `ON CONFLICT DO NOTHING` and skip processing altogether—leaving half the members without actual message delivery.

3. Database Write Locks / Deadlocks under Concurrent Parallel Batches:

- Splitting 22,400 members into concurrent parallel worker threads using `INSERT ... ON CONFLICT DO NOTHING` on un-ordered batches can lead to foreign key or primary key index locks and potential deadlocks in PostgreSQL if concurrent workers attempt to insert overlapping key ranges.

Recommended Full Database Schema

```
-- Locals (Tenants)
CREATE TABLE locals (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Members
CREATE TABLE members (
    id BIGSERIAL PRIMARY KEY,
    local_id BIGINT NOT NULL REFERENCES locals(id) ON DELETE CASCADE,
    full_name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    classification VARCHAR(100) NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'active',
    role VARCHAR(50) NOT NULL DEFAULT 'member' CHECK (role IN ('member', 'leadership')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_members_local_role ON members(local_id, role);

-- Announcements
CREATE TABLE announcements (
    id BIGSERIAL PRIMARY KEY,
    local_id BIGINT NOT NULL REFERENCES locals(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    body TEXT NOT NULL,
    needs_ack BOOLEAN NOT NULL DEFAULT FALSE,
    sent_at TIMESTAMPTZ NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_announcements_local ON announcements(local_id);

-- Recipient Delivery & Response Tracking
CREATE TABLE announcement_recipients (
    id BIGSERIAL PRIMARY KEY,
```

```
announcement_id BIGINT NOT NULL REFERENCES announcements(id) ON DELETE CASCADE,
member_id BIGINT NOT NULL REFERENCES members(id) ON DELETE CASCADE,

-- Delivery & Lifecycle States
delivery_status VARCHAR(50) NOT NULL DEFAULT 'pending'
    CHECK (delivery_status IN ('pending', 'sent', 'failed', 'undeliverable')),
sent_at TIMESTAMPTZ NULL,
read_at TIMESTAMPTZ NULL,
acknowledged_at TIMESTAMPTZ NULL,

-- RSVP States
rsvp VARCHAR(20) NOT NULL DEFAULT 'no_response'
    CHECK (rsvp IN ('coming', 'cant', 'no_response')),
rsvp_updated_at TIMESTAMPTZ NULL,

-- Idempotency Guarantee
CONSTRAINT uq_announcement_member UNIQUE (announcement_id, member_id)
);

-- Indexes for Fast Aggregation & Status Queries
CREATE INDEX idx_recipients_announcement_rsvp
    ON announcement_recipients(announcement_id, rsvp);

CREATE INDEX idx_recipients_dispatch_status
    ON announcement_recipients(announcement_id, delivery_status);
```
